Operating Systems CMPSC 473

Threads

February 5, 2009 - Lecture 7

Instructor: Trent Jaeger

- Last class:
  - Thread Background
- Today:
  - Thread Systems

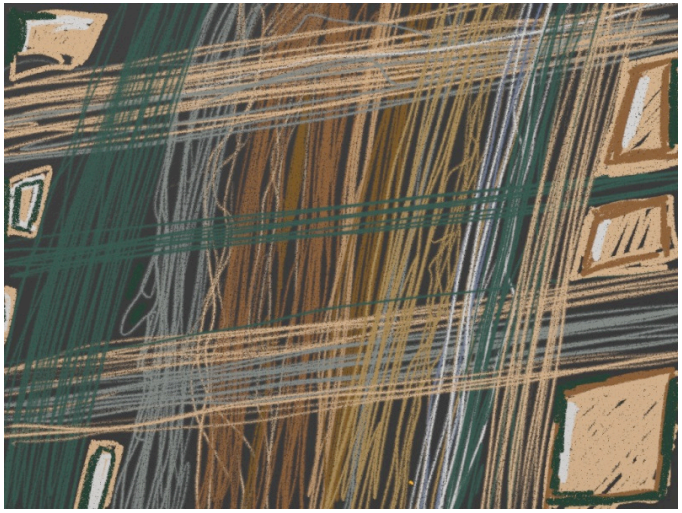
Threading Systems

What kind of problems would you solve with threads?

- Imagine you are building a web server
  - You could allocate a pool of threads, one for each client
    - Thread would wait for a request, get content file, return it
  - *How would the different thread models impact this?*
- Imagine you are building a web browser
  - You could allocate a pool of threads
    - Some for user interface
    - Some for retrieving content
    - Some for rendering content
  - *What happens if the user decided to stop the request?*
    - Mouse click on the stop button

Linux Threads

- Linux uses a one-to-one thread model
  - Threads are calls *tasks*
- Linux views threads are "contexts of execution"
  - Threads are defined separately from processes
  - I.e., a thread is assigned an address space



Linux Threads

- Linux system call
  - **clone(***int (*fn)(), void **stack, int flags, int argc, … /*args */***)**
  - Create a new thread (Linux task)
- May be created in the same address space or not
  - **Flags:** Clone VM, Clone Filesystem, Clone Files, Clone Signal Handlers
    - If clone with all these flags off, what system call is clone equal to?

Linux Threads

- Linux system call
  - **clone(***int (*fn)(), void **stack, int flags, int argc, … /*args */***)**
  - Create a new thread (Linux task)
- **clone**(*start*, *stack*, 0, 1)
  - What are *start* and *stack* for?
- So, clone provides significant flexibility over **fork**
  - Compare to **fork** variants in Solaris
- Example: http://tldp.org/FAQ/Threads-FAQ/clone.c

POSIX Threads

- POSIX Threads or Pthreads is a thread API specification
  - Not directly an implementation
  - Could be mapped to libraries or system calls
- Supported by Solaris and Linux

POSIX Threads

- Interface
- `pthread_create()`
  - *thread ID* {of the new thread}
    - Set on return
  - *attributes* {of the new thread}
    - stack size, scheduling information, etc.
  - *function* {one arg only}
    - the start function
  - *arg* {for function}
    - the start function
  - Return value, status {of the system call}

POSIX Threads

- `pthread_self()`
  - return thread ID
- `pthread_equal()`
  - for comparisons of thread ID's
- `pthread_exit()`
  - or just return from the start function
- `pthread_join()`
  - wait for another thread to terminate·retrieve value from `pthread_exit()`
- `pthread_cancel()`
  - terminate a thread, by TID
- `pthread_detach()`
  - thread is immune to join or cancel·runs independently until it terminates
- `pthread_attr_init()`
  - thread attribute modifiers

POSIX Threads

- Some example code

http://www.cse.psu.edu/~dheller/cse411/programs/thread_sample.c

POSIX Threads FAQ

- How do we pass multiple arguments to start a thread?
  - Build a struct and pass a pointer to it
- Is the **pthreads** id unique to the system?
  - No, just process -- Linux task ids are system-wide
- After **pthread_create**, which thread is running?
  - Like fork
- How many threads does **exit** terminate? **pthread_exit**?
  - All in process; only caller
- How are variables shared by threads?
  - Globals, local **static**, dynamic data (heap)

Inter-Thread Communication

- Can you use shared memory?
  - Already have it
  - Just need to allocate memory in the address space
    - No need for shm
    - Programming to pipes provides abstraction
- Can you use message passing?
  - Sure
  - Would have to build infrastructure
- An advantage of using kernel threads (e.g., Linux) is that system IPC can be used between threads
  - Would want optimized paths for common address space

Threading Issues

Fork/Exec Issues

- Semantics are ambiguous for multithreaded processes
- **fork()**
  - How does it interact with threads?
- **exec()**
  - What happens to the other threads?
- **fork**, then **exec**
  - Should all threads be copied?

Thread Cancellation

- So, you want to stop a thread from executing
  - Don't need it anymore
    - Remember the browser 'stop' example
- Two choices
  - Asynchronous cancellation
    - Terminate it now
    - What about resources allocated to the thread?
  - Deferred cancellation
    - Terminate when the thread is ready
    - Thread checks for termination periodically
    - **pthread_cancel**(thread_id)

Signal Handling

- What's a signal?
  - A form of IPC
  - Send a particular signal to another process
- The receiver's *signal handler* processes the signal on receipt
- Example
  - Tell the Internet daemon (**inetd**) to reread its config file
  - Send signal to **inetd**: kill -SIGHUP <pid>
  - **inetd**'s signal handler for the SIGHUP signal re-reads the config file
- Note: some signals cannot be handled by the receiving process, so they cause default action (kill the process)

Signal Handling

- Synchronous Signals
  - Generated by the kernel for the process
  - E.g., due to an exception -- divide by 0
    - Events caused by the thread receiving the signal
- Asynchronous Signals
  - Generated by another process
- Asynchronous signals are more difficult for multithreading

Signal Handling and Threads

- So, you send a signal to a process
  - Which thread should it be delivered to?
- Choices
  - Thread to which the signal applies
  - Every thread in the process
  - Certain threads in the process
  - A specific signal receiving thread
- It depends…

Signal Handling and Threads

- Synchronous vs. Asynchronous Cases
- Synchronous
  - Signal is delivered to the same process that caused the signal
  - Which thread(s) would you deliver the signal to?
- Asynchonous
  - Signal generated by another process
  - Which thread(s) in this case?

Thread Pools

- Problem: setup time
- Faster than setting up a process, but what is necessary?
  - How do we improve performance?

Thread Pools

- Pool of threads
  - Create (all) at initialization time
  - Assign task to a waiting thread
    - It's already made
  - Use all available threads
- What about when that task is done?
  - Suppose another request is in the queue…
  - Should we use running thread or another thread?

Scheduling

- So how many kernel threads should be available for a process?
  - In M:N model
- Suppose the last kernel thread for an application is to be blocked
  - Recall the relationship between kernel and user threads
  - What happens?

Scheduling

- Wouldn't it be nice if the kernel told the application and the application had a way to get more kernel threads?
  - *Scheduler activation*
    - At thread block, the kernel tells the application via an *upcall*
    - Aside: An upcall is a general term for an invocation of application function from the kernel
  - Application can then get a new thread created
    - See *lightweight threads* in Section 4.4.6

Reentrance and Thread-Safety

- Terms that you might hear
- *Reentrant Code*
  - Code that can be run by multiple threads concurrently
- *Thread-safe Libraries*
  - Library code that permits multiple threads to invoke the safe function
- Requirements
  - Rely only on input data
    - Or some thread-specific data

Why not threads?

- Threads can interfere with one another
  - Impact of more threads on caches
  - Impact of more threads on TLB
- Execution of multiple may slow them down
  - Impact of single thread vs. switching among threads
- Harder to program a multithreaded program
  - Multitasking hides context switching
  - Multithreading introduces concurrency issues

Summary

- Threads
  - Programming systems
  - Multi-threaded design issues
- Useful, but not a panacea
  - Slow down system in some cases
  - Can be difficult to program

- Next time: CPU Scheduling